Estrutura alvo de pastas do frontend e plano de quebra do `Index.tsx`

Objetivo
Definir uma estrutura alvo de pastas para o frontend do Ethos e um plano prÃ¡tico para decompor o `Frontend/src/pages/Index.tsx`, reduzindo acoplamento e preparando a aplicaÃ§Ã£o para a migraÃ§Ã£o local-first.

---

1. Problema atual

O `Frontend/src/pages/Index.tsx` concentra responsabilidades demais:
- splash e intro
- login versus app autenticado
- resoluÃ§Ã£o de pÃ¡gina atual
- navegaÃ§Ã£o interna
- prÃ©-carregamento de pÃ¡ginas
- seleÃ§Ã£o de paciente e sessÃ£o
- renderizaÃ§Ã£o condicional por papel e contexto

Isso gera:
- alto acoplamento
- manutenÃ§Ã£o difÃcil
- testes difÃceis
- evoluÃ§Ã£o lenta por domÃnio
- dificuldade para introduzir persistÃªncia local como fonte primÃ¡ria

---

2. PrincÃpios da nova estrutura

2.1 Separar por domÃnio
Cada Ã¡rea principal do produto deve evoluir em mÃ³dulo prÃ³prio.

2.2 Separar app shell de domÃnio
Layout, navegaÃ§Ã£o e bootstrap nÃ£o devem viver misturados com regras clÃnicas.

2.3 Separar UI state de domain state
SeleÃ§Ã£o de tela e estado visual nÃ£o devem virar a base do domÃnio clÃnico.

2.4 Separar local de remoto
A nova estrutura deve preparar:
- repositories locais
- adapters remotos opcionais
- services de domÃnio que orquestram os dois

2.5 Permitir migraÃ§Ã£o incremental
A estrutura precisa permitir mover domÃnio por domÃnio sem reescrever tudo.

---

3. Estrutura alvo de pastas

```text
Frontend/src/
  app/
    App.tsx
    AppBootstrap.tsx
    providers/
      AppProviders.tsx
      QueryProvider.tsx
      ThemeProvider.tsx
    routing/
      PageResolver.tsx
      NavigationController.ts
      pageRegistry.ts
      pageRedirects.ts
    shell/
      AppShell.tsx
      AppFrame.tsx
      AppSidebar.tsx
      AppHeader.tsx

  modules/
    auth/
      components/
```

```
    pages/
    hooks/
    services/
    contexts/
    local/
    remote/

  patients/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  sessions/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  records/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  documents/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  reports/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  agenda/
    components/
    pages/
    hooks/
    services/
    repositories/
    models/
    local/
    remote/

  account/
    components/
    pages/
    hooks/
    services/

  admin/
```

```
    components/
    pages/
    hooks/
    services/

domain/
  shared/
    types/
    errors/
    value-objects/
    contracts/

infrastructure/
  local-db/
    db.ts
    migrations/
    stores/
    adapters/
  remote/
    apiClient.ts
    controlClient.ts
    connectors/
  crypto/
    encryption.ts
    key-management.ts
  backup/
    export.ts
    restore.ts

state/
  ui/
    appUiStore.ts
    navigationStore.ts
    connectivityStore.ts
  runtime/
    featureFlags.ts

  components/
    ui/
    shared/

  hooks/
    shared/

  lib/
    utils/
    formatting/
    validation/

  config/
    runtime.ts
    env.ts
```

---

4. Regras de responsabilidade por camada

`app/`
ResponsÃ¡vel por:
- bootstrap da aplicaÃ§Ã£o
- providers globais
- shell e estrutura principal
- roteamento e resoluÃ§Ã£o de pÃ¡gina

NÃ£o deve conter:
- lÃ³gica clÃnica de domÃnio
- acesso direto a dados clÃnicos

`modules/`
ResponsÃ¡vel por:
- tudo que pertence a uma Ã¡rea funcional do produto
- pÃ¡ginas, componentes, hooks, services e repositories de cada domÃnio

`domain/shared/`
ResponsÃ¡vel por:
- contratos e tipos centrais compartilhados
- regras comuns entre mÃ³dulos

`infrastructure/local-db/`
ResponsÃ¡vel por:
- banco local
- migrations
- adapters de persistÃªncia

`infrastructure/remote/`
ResponsÃ¡vel por:
- comunicaÃ§Ã£o remota
- integraÃ§Ã£o com APIs online
- gateways conectados

`state/ui/`
ResponsÃ¡vel por:
- estado transitÃ³rio da interface
- navegaÃ§Ã£o atual
- seleÃ§Ã£o visual
- flags de UI

---

5. O que sai do `Index.tsx`

O `Index.tsx` atual deve deixar de concentrar estas responsabilidades.

Sai do `Index.tsx`
- splash flow
- intro splash
- login versus app autenticado
- resoluÃ§Ã£o da pÃ¡gina atual
- seleÃ§Ã£o de sessÃ£o
- seleÃ§Ã£o de paciente
- renderizaÃ§Ã£o condicional gigante por `switch`
- prefetch por papel
- navegaÃ§Ã£o interna centralizada

Fica fora dele em componentes dedicados
- bootstrap do app
- auth gate
- page resolver
- navigation controller
- shell autenticado
- mÃ³dulos por domÃnio

---

6. Estrutura alvo de decomposiÃ§Ã£o do `Index.tsx`

Estado atual
Hoje ele mistura:
- app boot
- auth flow
- page orchestration
- state machine de navegaÃ§Ã£o
- renderizaÃ§Ã£o de pÃ¡ginas

Estrutura futura sugerida

`AppBootstrap.tsx`
ResponsÃ¡vel por:
- decidir quando mostrar splash
- inicializar providers e readiness local
- entregar para a camada certa: autenticada ou nÃ£o

`AuthGate.tsx`
ResponsÃ¡vel por:
- decidir entre fluxo local desbloqueado, login online opcional, ou tela pÃºblica
- separar autenticaÃ§Ã£o local de conta online no futuro

`AuthenticatedApp.tsx`
ResponsÃ¡vel por:
- montar shell autenticado
- injetar page resolver
- acoplar navegaÃ§Ã£o com contexto autenticado

`PageResolver.tsx`
ResponsÃ¡vel por:
- resolver qual pÃ¡gina renderizar
- parar de usar um `switch` gigante concentrado no `Index.tsx`

`NavigationController.ts`
ResponsÃ¡vel por:
- normalizar navegaÃ§Ã£o
- aplicar redirects
- lidar com estado de pÃ¡gina atual
- permitir migraÃ§Ã£o futura para navegaÃ§Ã£o mais modular

`SelectionController` ou stores equivalentes
ResponsÃ¡vel por:
- sessÃ£o selecionada
- paciente selecionado
- estado transitÃ³rio de contexto de navegaÃ§Ã£o

---

7. Plano de quebra do `Index.tsx` por etapas

Etapa 1 â€” Extrair boot e splash
Criar
- `app/AppBootstrap.tsx`
- `app/AuthGate.tsx`

Mover para lÃ¡
- splash logic
- logo reveal logic
- escolha inicial entre login e app autenticado

Resultado
O `Index.tsx` deixa de cuidar do inÃcio do ciclo de vida do app.

Etapa 2 â€” Extrair navegaÃ§Ã£o e resoluÃ§Ã£o de pÃ¡gina
Criar
- `app/routing/PageResolver.tsx`
- `app/routing/NavigationController.ts`

Mover para lÃ¡
- `currentPage`
- redirects
- `handleNavigate`
- `handlePrefetch`
- `renderPageContent`

Resultado
A resoluÃ§Ã£o das pÃ¡ginas deixa de ficar toda concentrada em um componente monolÃtico.

Etapa 3 â€” Extrair seleÃ§Ã£o contextual
Criar
- `state/ui/navigationStore.ts` ou mÃ³dulo dedicado

Mover para lÃ¡
- `selectedSessionId`
- `selectedPatientId`
- handlers como:
  - `handleSessionClick`
  - `handleBackFromSession`
  - `handleOpenPatient`
  - `handleBackFromPatient`
  - `handleOpenProntuario`

Resultado
O contexto de navegaÃ§Ã£o deixa de ser estado local espalhado dentro do antigo `Index.tsx`.

Etapa 4 â€” Extrair o app autenticado
Criar
- `app/AuthenticatedApp.tsx`
- `app/UnauthenticatedApp.tsx`

Mover para lÃ¡
- renderizaÃ§Ã£o do `AppShell`
- renderizaÃ§Ã£o do `LoginPage`
- integraÃ§Ã£o com `ProtectedPage`

Resultado
A distinÃ§Ã£o entre â€œapp logadoâ€ e â€œentrada nÃ£o autenticadaâ€ vira uma fronteira explÃcita.

Etapa 5 â€” Mover pÃ¡ginas para mÃ³dulos por domÃnio
Exemplo
- `PatientsPage` vai para `modules/patients/pages/PatientsPage.tsx`
- `PatientDetailPage` vai para `modules/patients/pages/PatientDetailPage.tsx`
- `SessionPage` vai para `modules/sessions/pages/SessionPage.tsx`
- `ProntuarioPage` vai para `modules/records/pages/ProntuarioPage.tsx`
- `ReportsPage` vai para `modules/reports/pages/ReportsPage.tsx`

Resultado
Cada domÃnio ganha independÃªncia arquitetural real.

---

8. Ordem prÃ¡tica recomendada

Fase 1
- criar `app/`
- criar `AppBootstrap.tsx`
- criar `AuthGate.tsx`
- extrair splash e auth entry

Fase 2
- criar `routing/`
- extrair `PageResolver`
- extrair `NavigationController`

Fase 3
- criar `state/ui/navigationStore.ts`
- mover seleÃ§Ã£o de sessÃ£o/paciente

Fase 4
- criar mÃ³dulos `patients`, `sessions`, `records`
- mover pÃ¡ginas centrais

Fase 5
- introduzir repositories locais por domÃnio
- adaptar hooks para consumir domÃnio local-first

---

9. Mapeamento inicial sugerido

Mover primeiro
- `HomePage`
- `PatientsPage`
- `PatientDetailPage`
- `SessionPage`
- `ProntuarioPage`

Mover depois
- `ReportsPage`
- `DocumentsPage`
- `AgendaPage`
- `FinancePage`

- `ContractsPage`

Manter mais ao fim
- Ã¡reas admin
- test lab
- diagnostics
- pÃ¡ginas menos crÃticas para o fluxo clÃnico principal

---

10. CritÃ©rios de pronto

A estrutura alvo comeÃ§a a funcionar quando
- `Index.tsx` deixa de ser o principal orquestrador do produto
- shell, auth e page resolution passam a estar separados
- pelo menos pacientes e sessÃµes jÃ¡ vivem em mÃ³dulos prÃ³prios
- o frontend fica pronto para acoplar repositories locais

A quebra do `Index.tsx` pode ser considerada concluÃda quando
- ele deixar de conter `switch` central gigante
- deixar de controlar seleÃ§Ã£o clÃnica diretamente
- deixar de concentrar boot, auth e renderizaÃ§Ã£o de domÃnio

---

11. Riscos

Risco 1
Mover arquivos sem redefinir responsabilidades reais.

Risco 2
Criar mÃ³dulos por pasta, mas manter dependÃªncias cruzadas demais.

Risco 3
Acoplar a nova estrutura Ã API remota do mesmo jeito antigo.

Risco 4
Tentar migrar todos os domÃnios antes de estabilizar pacientes e sessÃµes.

---

12. PrÃ³ximos artefatos recomendados

Depois deste documento, os prÃ³ximos mais Ãºteis sÃ£o:
1. contrato dos repositories por domÃnio
2. plano de migraÃ§Ã£o de pacientes e sessÃµes para persistÃªncia local
3. checklist tÃ©cnico da etapa 1 de refactor
4. plano de revisÃ£o do `AuthContext` para separar auth local e conta online

---

Resumo final
O objetivo nÃ£o Ã© apenas â€œorganizar melhor as pastasâ€.

O objetivo Ã© criar uma estrutura que permita ao Ethos:
- modularizar o frontend
- reduzir acoplamento
- tornar persistÃªncia local a fonte primÃ¡ria
- tratar nuvem como opcional
- evoluir o produto sem reescrita total